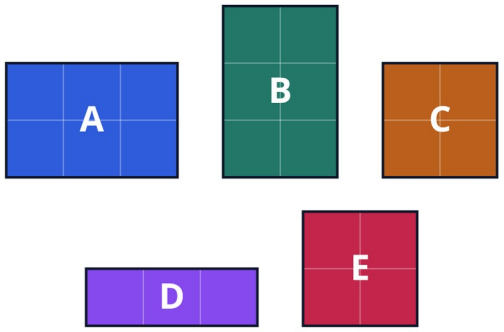
# Plan de masse par arbre B*

Cinq blocs, un cadre, et un arbre qui dit où va chaque bloc

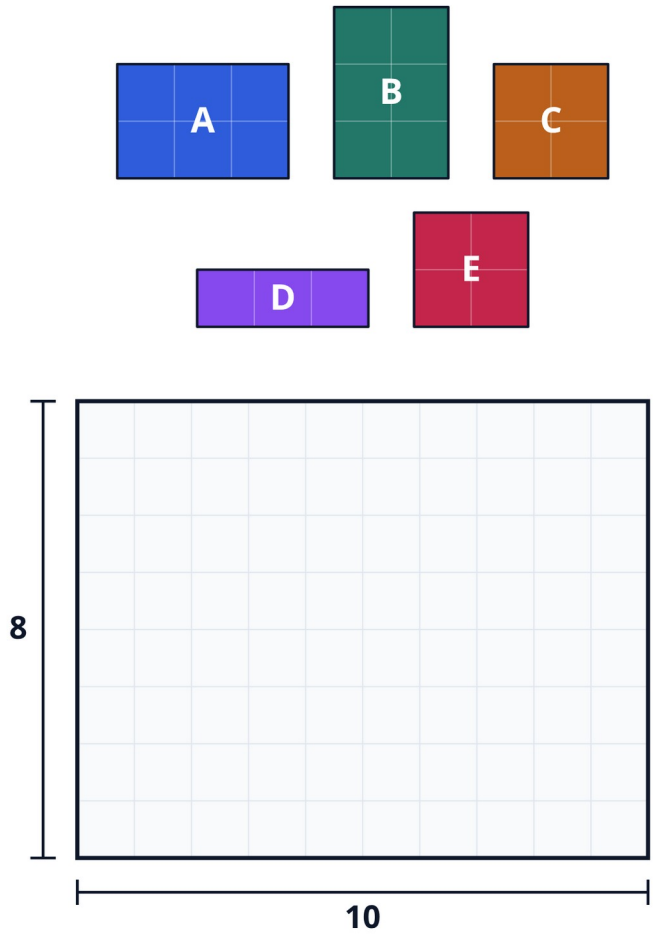
# Cinq blocs

- Une puce est faite de grandes pièces rectangulaires, appelées blocs
- Le nôtre a cinq blocs, nommés de A à E
- Ils sont sur du papier quadrillé : on peut compter chaque largeur et chaque hauteur



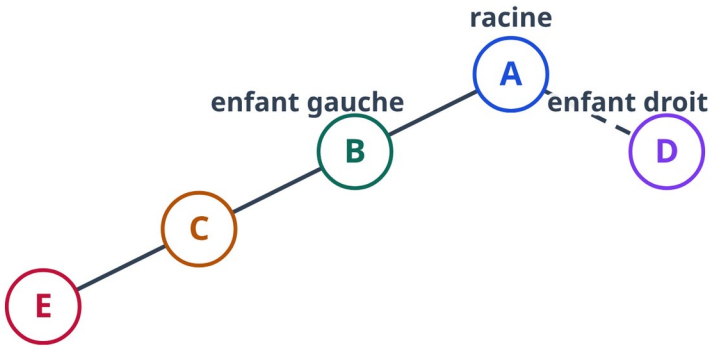
# Le contour

- Un plan de masse dit où se trouve chaque bloc sur la puce
- Chaque bloc doit rester dans un cadre appelé le contour
- Notre contour fait dix cases de large et huit cases de haut
- Plus les blocs sont serrés, plus la puce peut être petite



# Un arbre comme recette

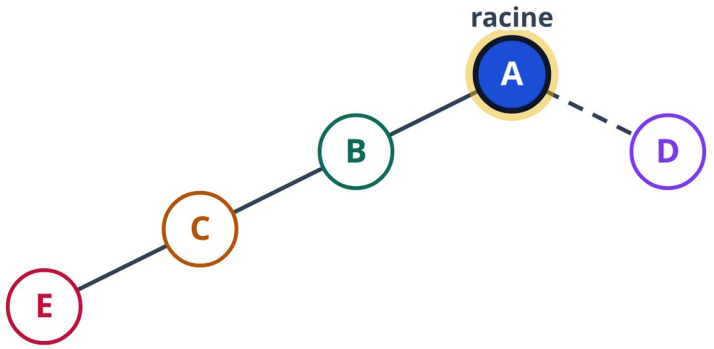
- Noter chaque position est long, alors nous utilisons une recette : un arbre B-étoile
- Chaque bloc est un nœud de l'arbre
- Le nœud du haut, A, s'appelle la racine
- Chaque nœud peut avoir deux enfants : un enfant gauche et un enfant droit

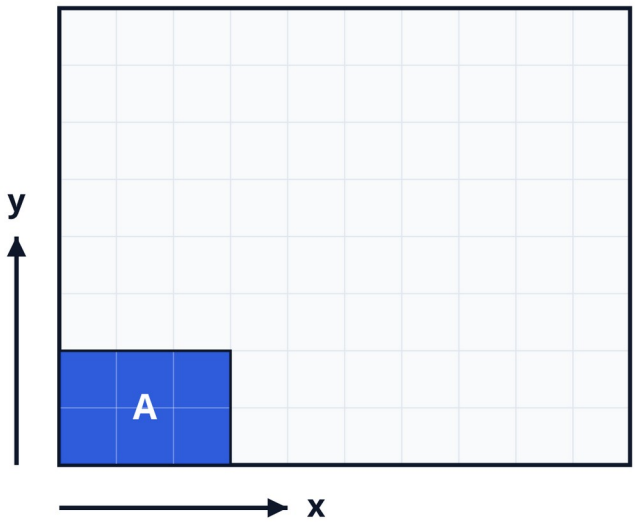





# La racine va dans le coin

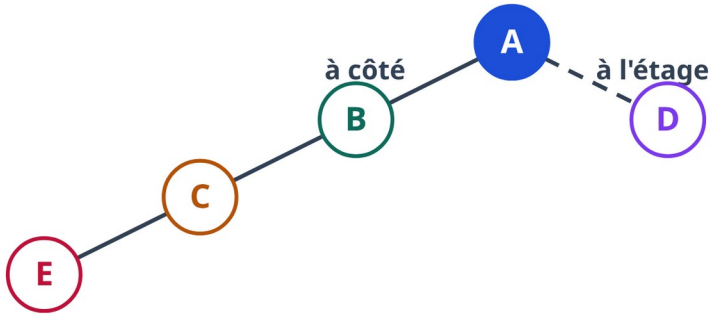
- La racine va dans le coin inférieur gauche du contour
- À partir de ce coin, x compte les cases vers la droite, et y compte les cases vers le haut

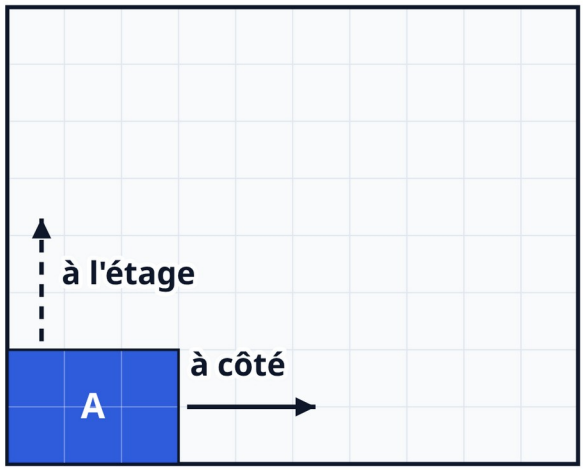


# À gauche, à côté ; à droite, à l'étage

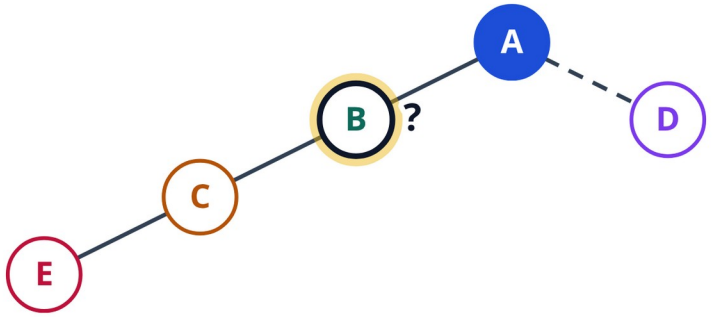
- Un enfant gauche va à côté : collé contre le côté droit de son parent
- Un enfant droit va à l'étage : sur son parent, avec le même bord gauche
- Les noms semblent inversés : gauche veut dire à côté, droite veut dire à l'étage
- Dans l'arbre, un trait plein veut dire à côté, un pointillé veut dire à l'étage

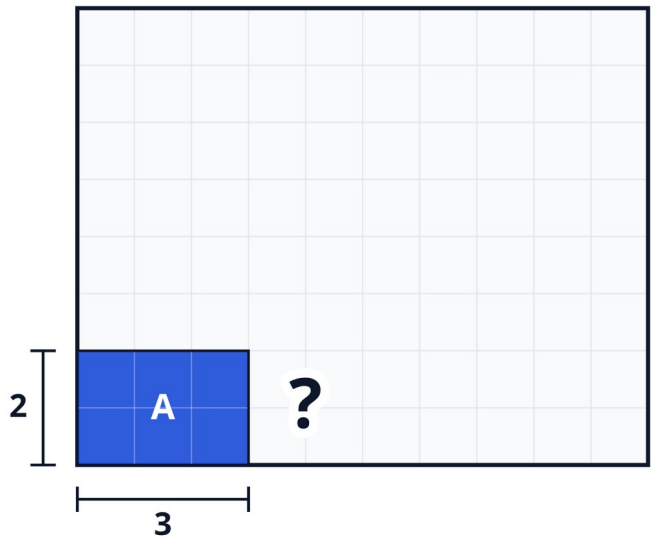


# Où va B ?

- A fait trois cases de large et deux cases de haut
- B est l'enfant gauche de A
- Faites une pause et devinez : où va le bord gauche de B ?



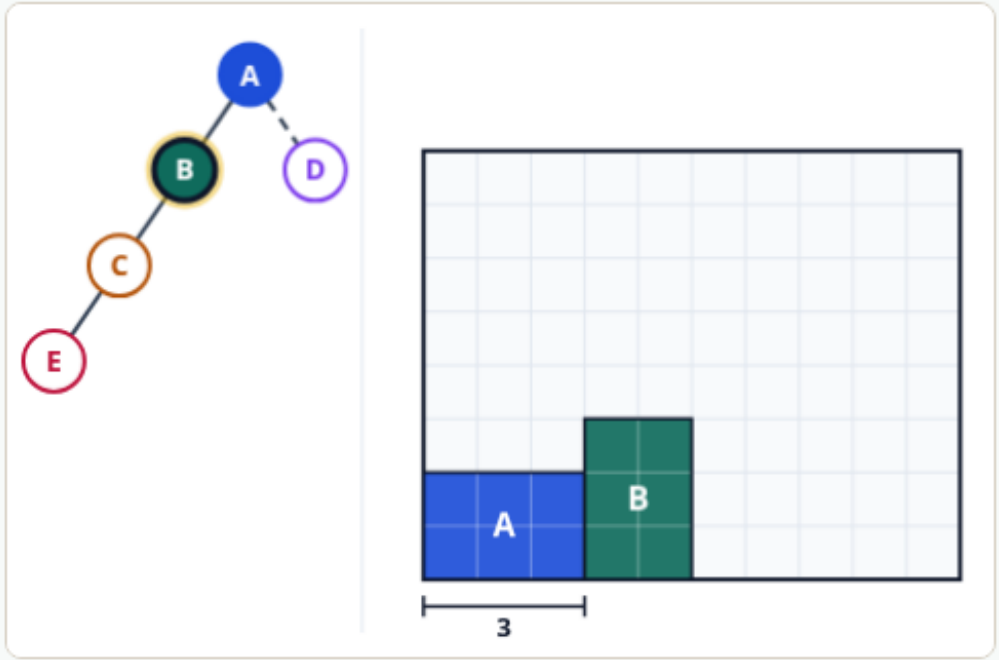



# B va à côté, en x = 3

ARBRE B*

## B va à côté, en x = 3

Étape 1 / 5 · root · module02-03-bstar-tree




### Explication

B va à côté, en x égal trois : A commence à zéro, et A fait trois de large. Rien n'est sous B, donc il repose sur le sol, en y égal zéro.

- À côté de A
- x = 0 + 3 = 3
- Sur le sol : y = 0

```
B en x = 3, y = 0
```

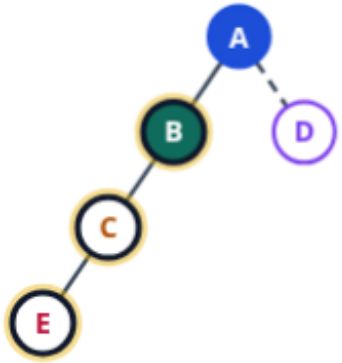
B va à côté, en x = 3

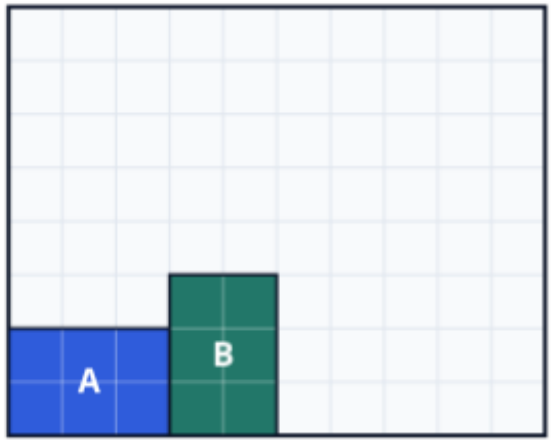
# Le parcours : A, B, C, E, puis D

ARBRE B*

## Le parcours : A, B, C, E, puis D

Étape 2 / 5 · left-chain · module02-03-bstar-tree




### Explication

On place les blocs un par un, dans un ordre de parcours fixe. On place un nœud, puis tout son côté gauche, et seulement ensuite son enfant droit. Le parcours donne donc A, B, C et E, et D vient en dernier.

- On place un nœud
- puis tout son côté gauche
- seulement ensuite son enfant droit

```
parcours : A, B, C, E, D
```

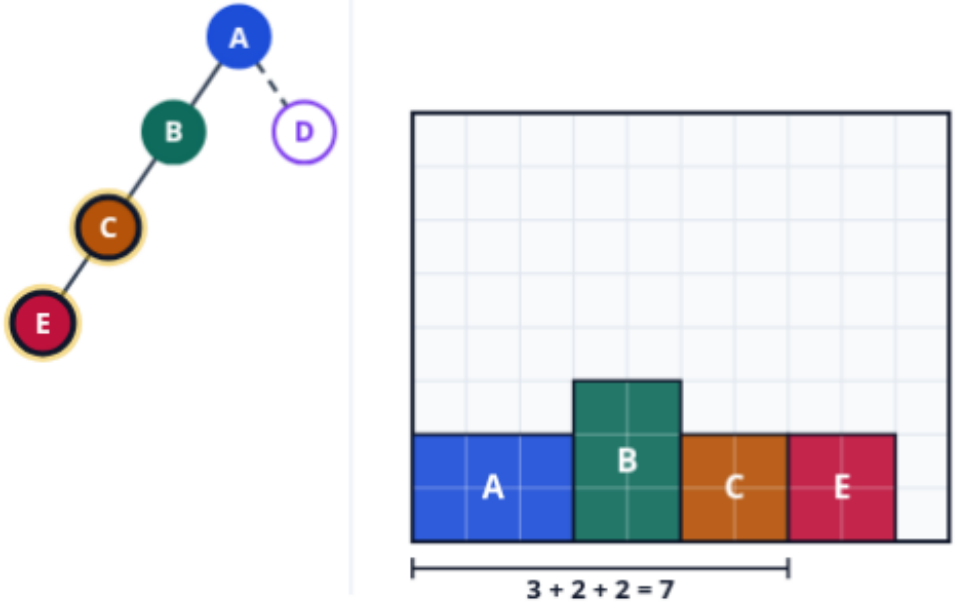
Le parcours : A, B, C, E, puis D

# C et E vont à côté

ARBRE B*

## C et E vont à côté

Étape 3 / 5 · right-d · module02-03-bstar-tree




### Explication

C est l'enfant gauche de B, donc il va à côté de B, en x égal cinq. E est l'enfant gauche de C, donc il arrive à côté de C, en x égal sept. Les largeurs s'additionnent : trois, plus deux, plus deux, font sept.

- C en x = 5
- E en x = 7
- 3 + 2 + 2 = 7

```
C en x = 5
E en x = 7
```

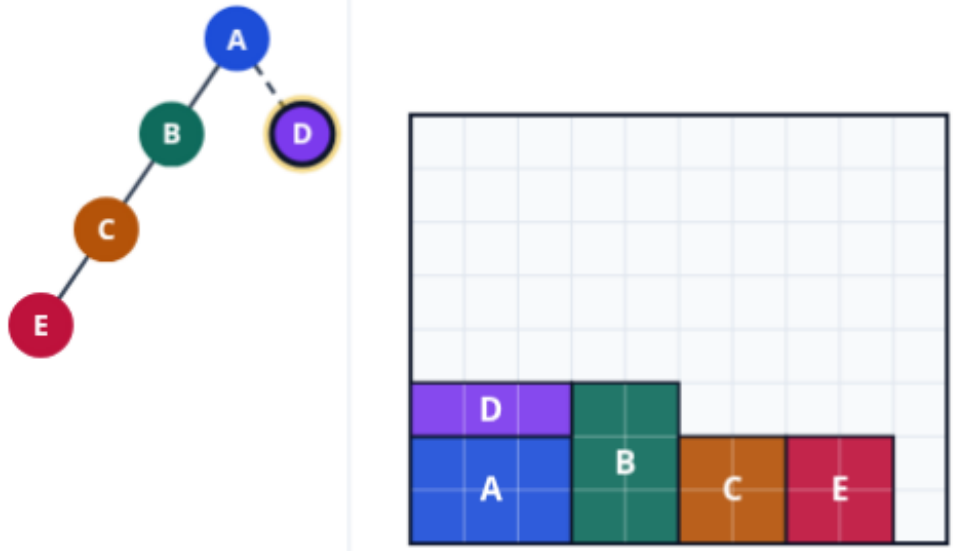
C et E vont à côté

# D monte à l'étage, sur A

ARBRE B*

## D monte à l'étage, sur A

Étape 4 / 5 · legal · module02-03-bstar-tree




### Explication

Le côté gauche est fini, donc le parcours revient à D, l'enfant droit de A. D monte à l'étage, donc il commence en x égal zéro, comme A. Il se pose sur A, en y égal deux.

- D : enfant droit de A
- À l'étage : x = 0
- Sur A : y = 2

```
D en x = 0, y = 2
```

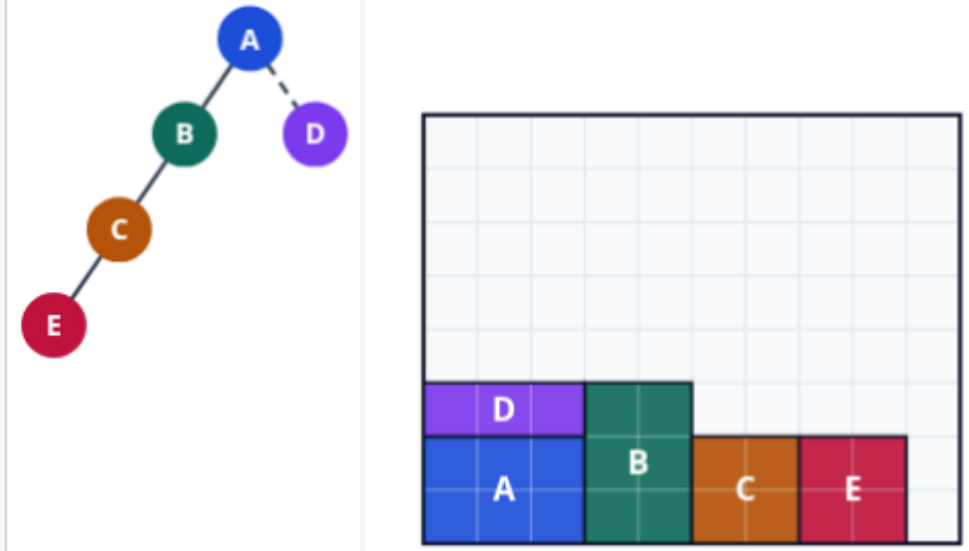
D monte à l'étage, sur A

# Cinq blocs placés

ARBRE B*

## Cinq blocs placés

Étape 5 / 5 · takeaway · module02-03-bstar-tree




### Explication

Les cinq blocs sont placés, et chacun est dans le contour. Faites une pause et devinez : si on échange A et C dans l'arbre, D arrive-t-il encore en y égal deux ?

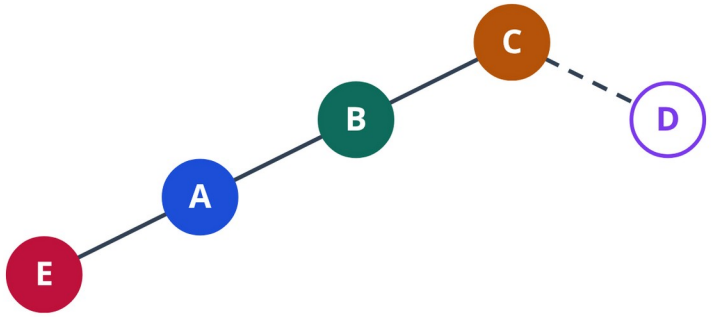
- Les cinq dans le contour
- On échange A et C :
- D est-il encore en y = 2 ?

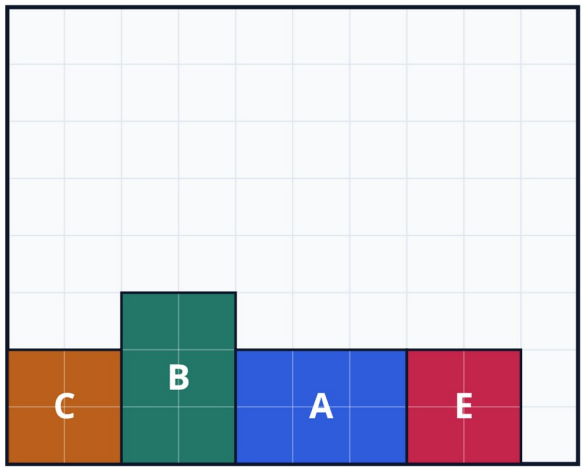
```
5 sur 5 placés
```

Cinq blocs placés

# Échanger A et C

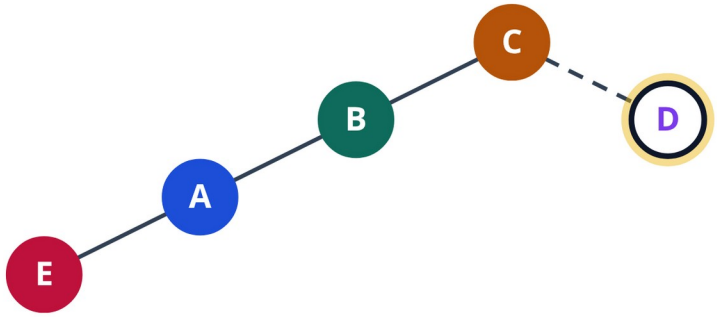
- Non : D arrive une case plus haut
- Voici l'arbre où A et C sont échangés : C est la racine, et A prend l'ancienne place de C
- C, B, A et E vont toujours à côté, l'un après l'autre

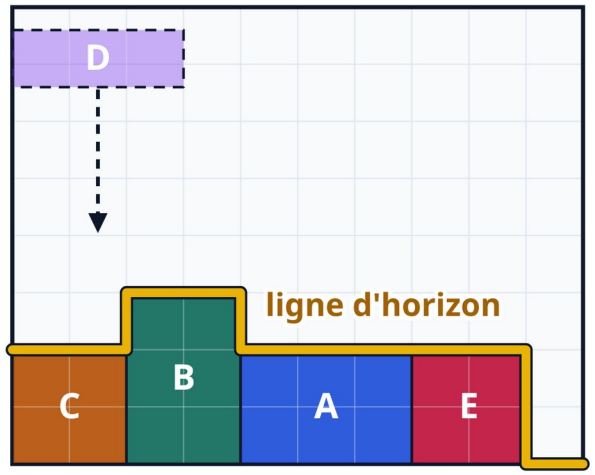


# La ligne d'horizon

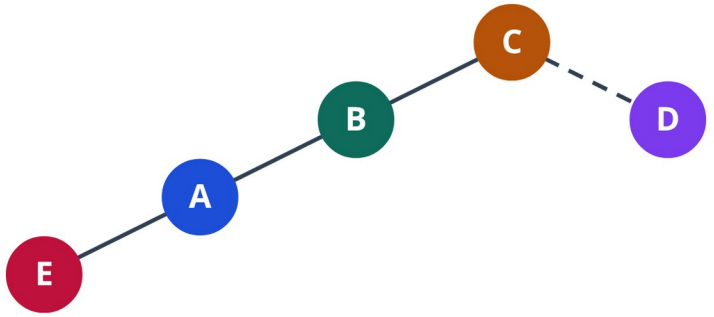
- Imaginez maintenant les blocs comme une ville, vue de loin
- Le bord supérieur de tout ce qui est déjà placé s'appelle la ligne d'horizon
- Un nouveau bloc tombe du ciel, comme au Tetris, jusqu'à toucher la ligne d'horizon

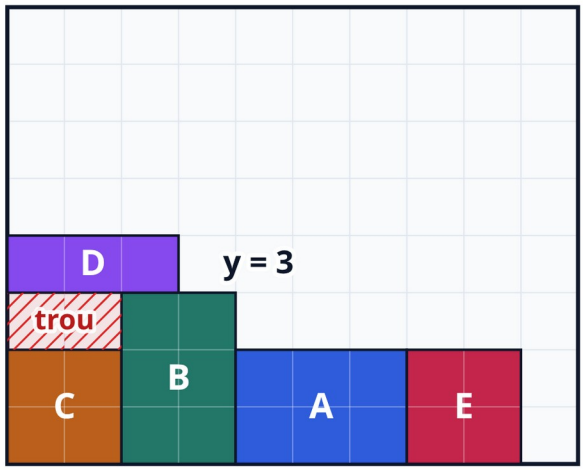


# D se pose sur B

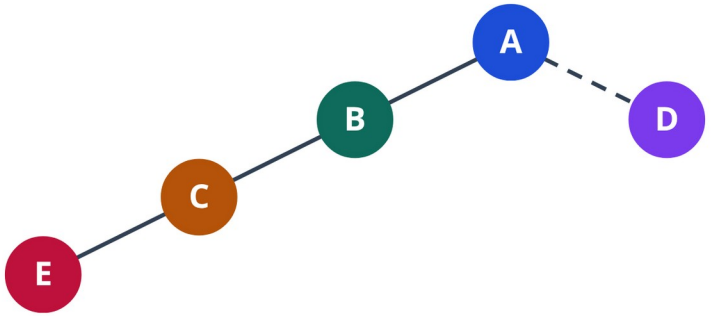
- D fait trois de large, donc il passe au-dessus de C et d'une partie de B
- B fait trois de haut, donc il arrête D en y égal trois
- Cela laisse un trou de deux cases vides au-dessus de C

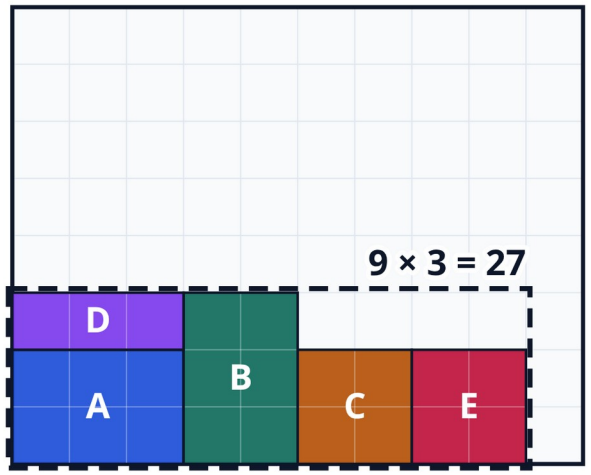


# La boîte englobante

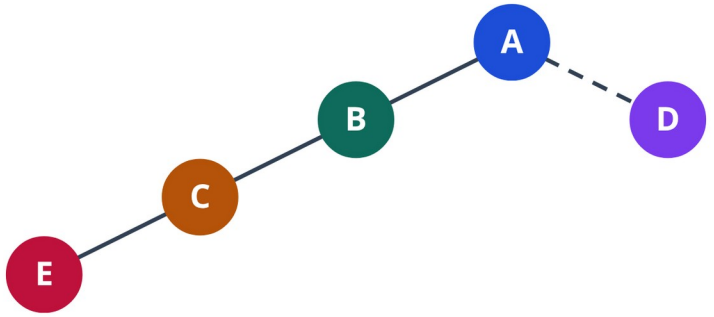
- Alors, quel plan de masse est le meilleur ?
- Pour comparer, on dessine la boîte englobante, le plus petit rectangle autour des blocs
- Pour le premier arbre, elle fait neuf de large et trois de haut : vingt-sept cases

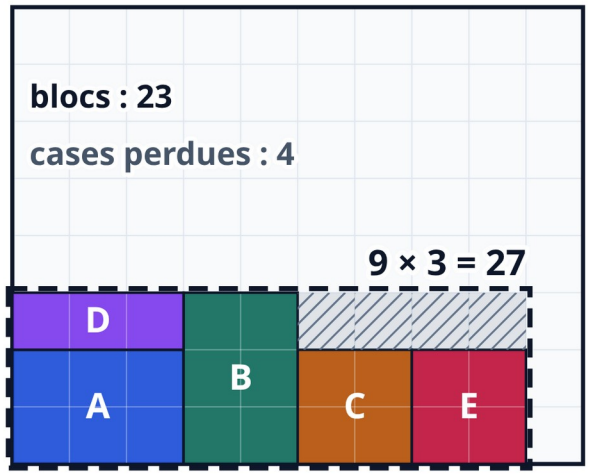


# Les cases perdues

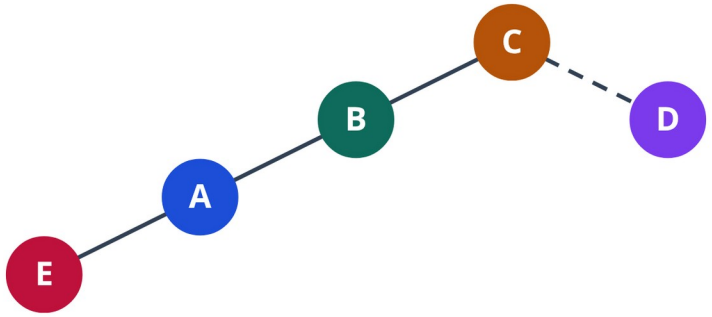
- Les cinq blocs couvrent vingt-trois cases
- Les quatre autres cases de la boîte sont vides : ce sont des cases perdues
- Moins il y a de cases perdues, plus le plan de masse est serré

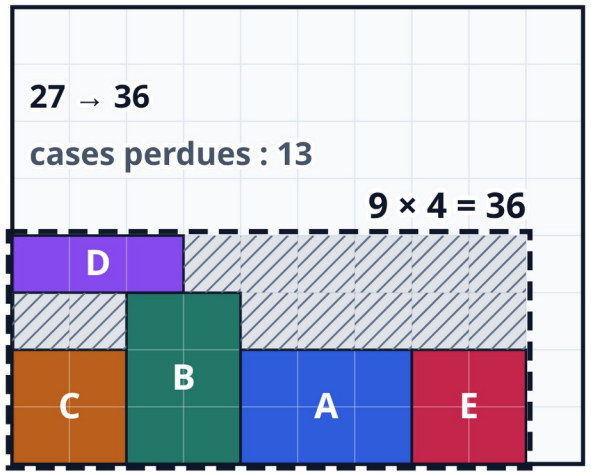


# Un échange, un score moins bon

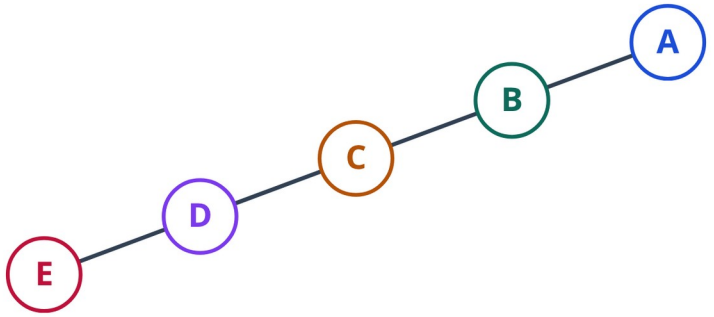
- L'arbre échangé a une boîte de neuf sur quatre : trente-six cases, treize perdues
- Un seul échange de deux noms a donc changé l'image et le score
- Vingt-sept est devenu trente-six : le premier arbre est meilleur

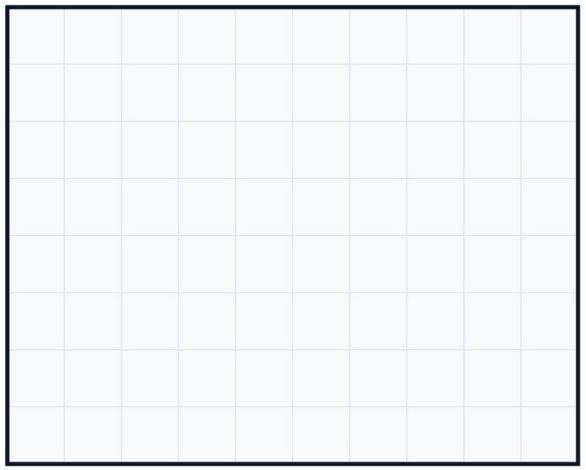


# Une longue rangée

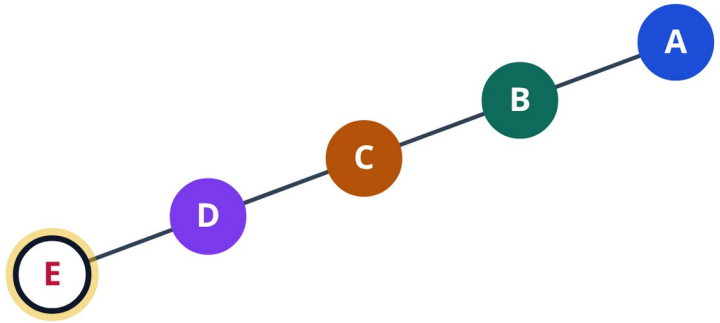
- Prenons maintenant un arbre où chaque enfant est un enfant gauche
- Faites une pause et devinez : cette longue rangée tient-elle dans le contour ?

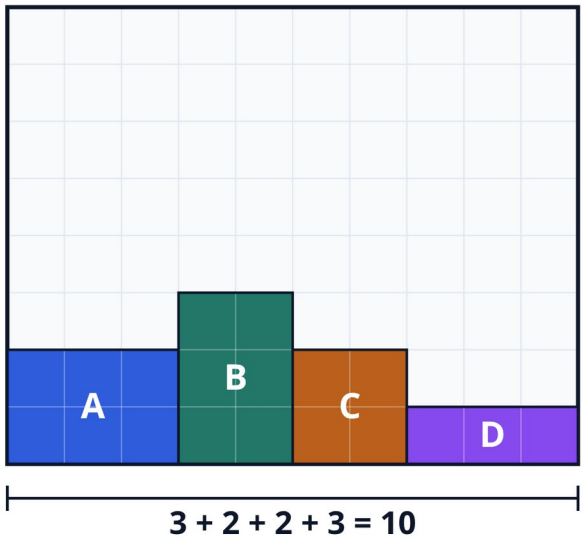


# Chaque bloc à côté

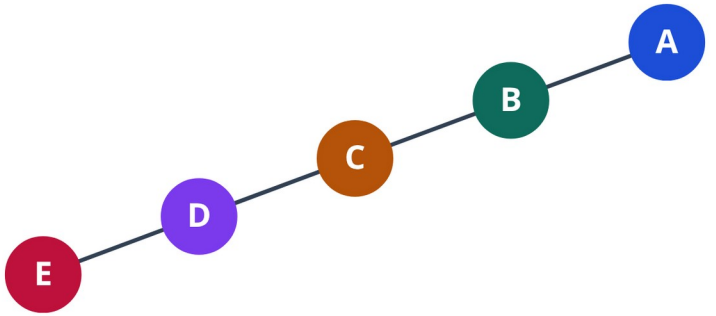
- Non, elle ne tient pas
- Chaque bloc va à côté : trois, deux, deux et trois font dix
- Le contour est déjà plein, et E attend encore

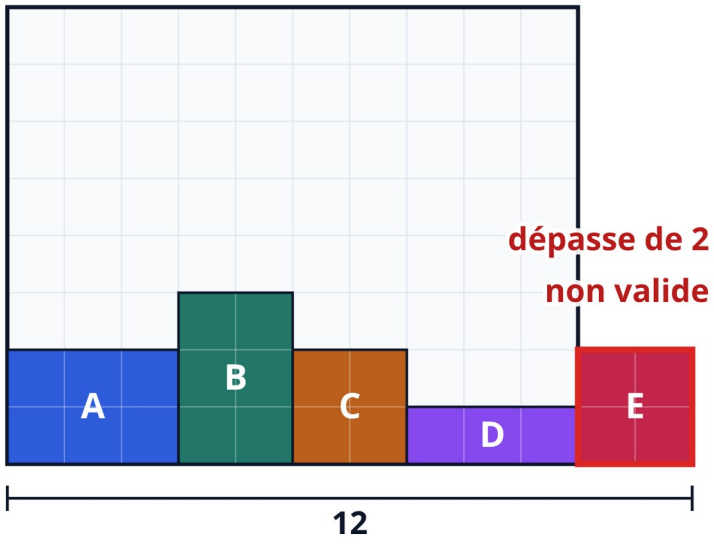


# Un bloc qui dépasse

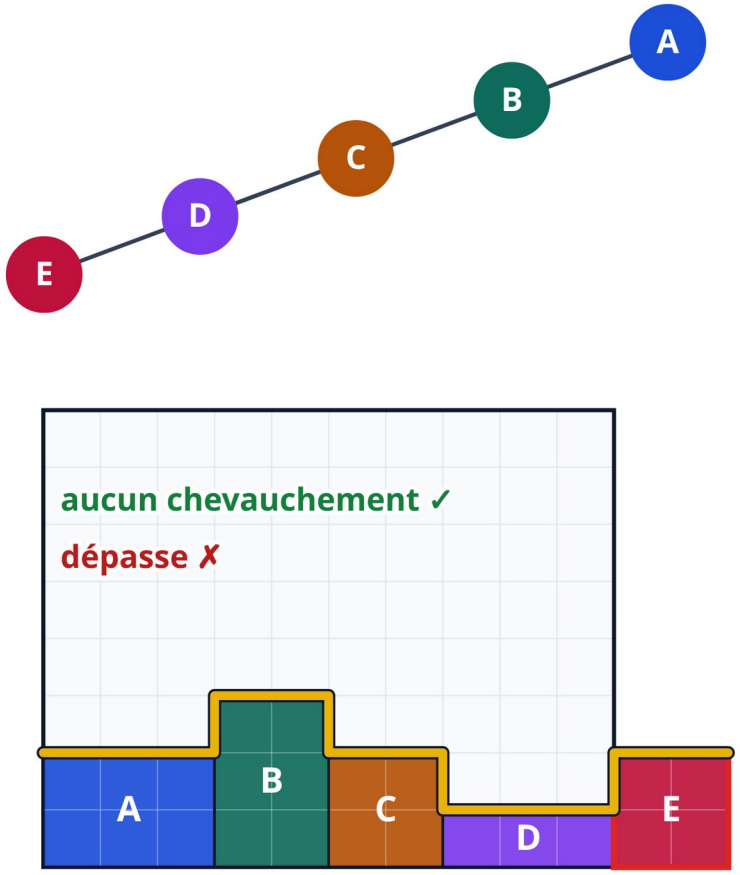
- E arrive en x égal dix, donc la rangée fait douze cases de large
- E dépasse du contour de deux cases
- Un plan de masse est valide seulement si chaque bloc est dans le contour
- Cette rangée n'est donc pas valide



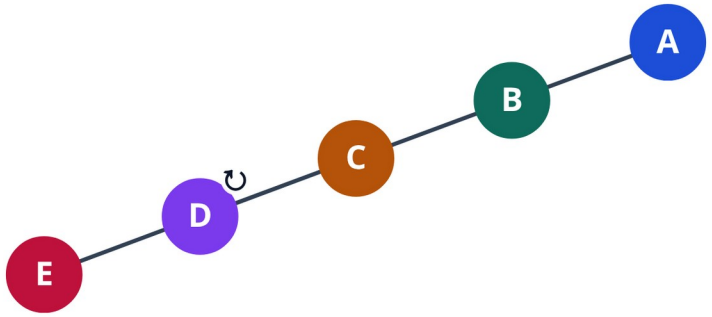
# Jamais de chevauchement

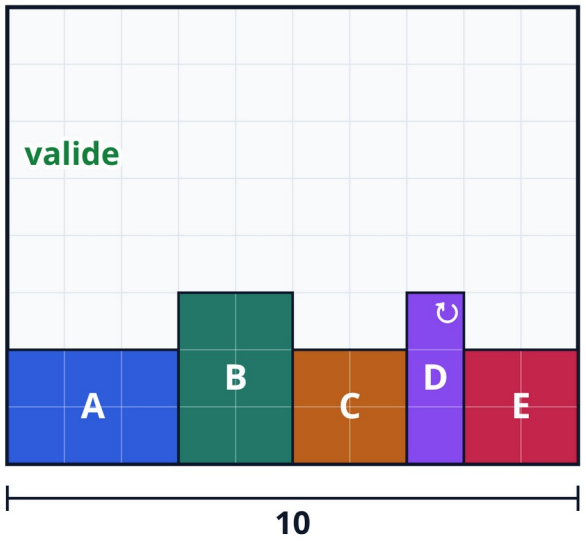
- Remarquez que les blocs ne se chevauchent jamais
- Chaque bloc tombe sur la ligne d'horizon, donc il ne peut jamais atterrir dans un autre
- Avec un arbre B-étoile, dépasser est la seule façon d'échouer



# Tourner un bloc

- On peut tourner un bloc sur le côté : sa largeur et sa hauteur s'échangent
- Tournez D, et il fait une case de large et trois de haut
- La rangée fait maintenant dix de large : elle tient pile, et elle est de nouveau valide



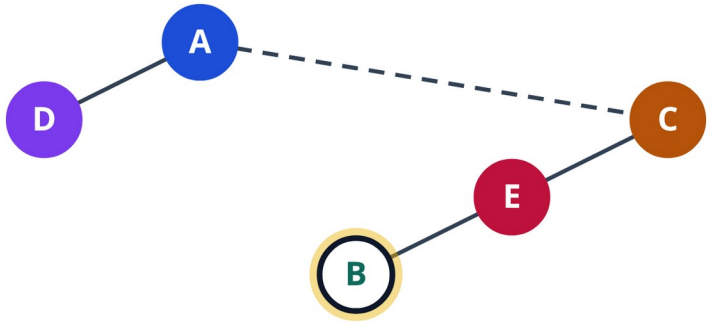



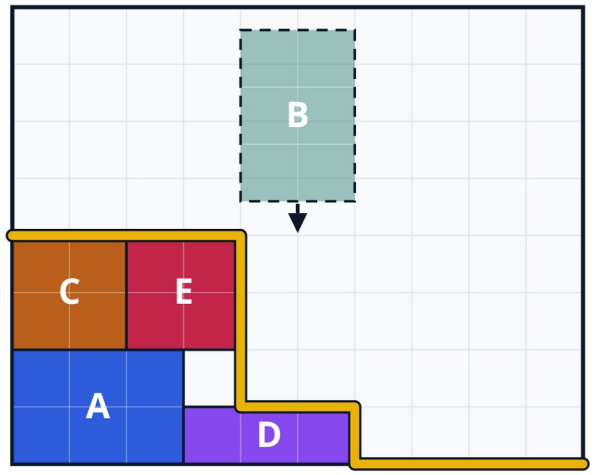
# Trois petits changements

- Tourner est l'un des trois petits changements
- On peut aussi échanger deux noms dans l'arbre, comme avec A et C
- Ou déplacer une feuille, un bloc sans enfant, vers une place libre de l'arbre
- Chaque changement donne un nouveau plan de masse, avec un nouveau score
- Faites une pause et devinez : un arbre peut-il battre vingt-sept ?
- Et quelle est la plus petite boîte ?

# Le plan de masse le plus serré

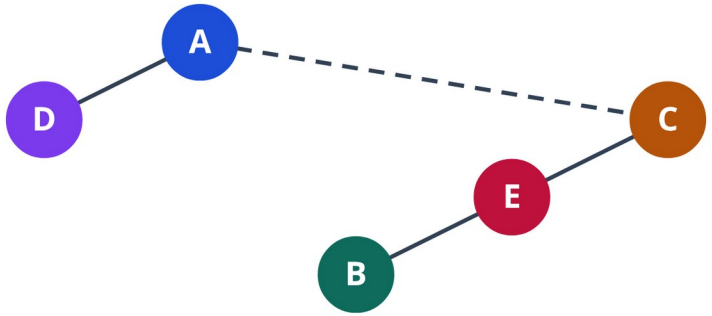
- Oui : beaucoup d'arbres battent vingt-sept
- Les meilleurs n'ont besoin que de vingt-quatre cases
- Voici l'un des meilleurs arbres
- Son parcours place A, D, C et E, et B vient en dernier

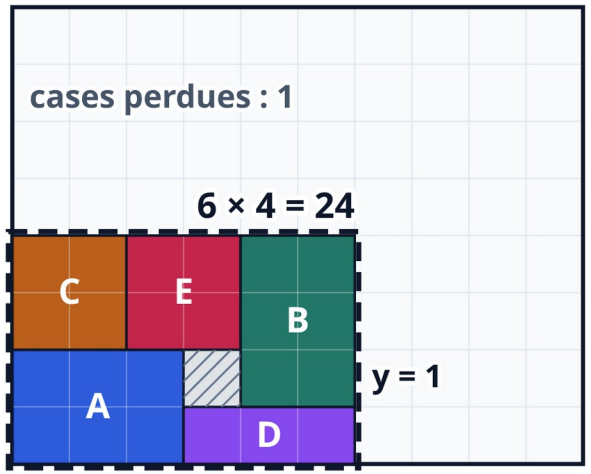


# B est soulevé par la ligne d'horizon

- Sous B, la ligne d'horizon est le haut de D, donc B arrive en y égal un
- Il comble le trou, et la boîte fait six de large et quatre de haut : vingt-quatre cases
- Une seule case est perdue



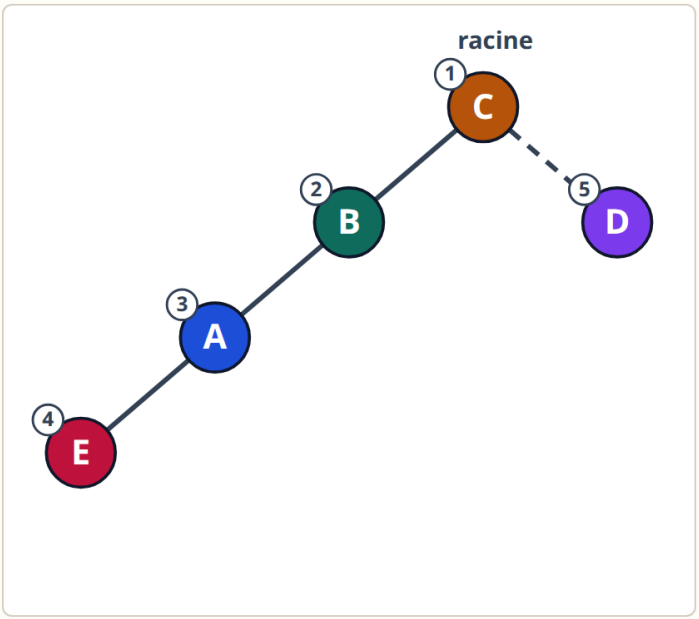
# Pourquoi pas vingt-trois ?

- Une boîte de vingt-trois cases, sans case perdue, pourrait-elle contenir les cinq blocs ?
- Vingt-trois cases ne forment qu'un rectangle épais d'une seule case
- Le bloc B est trop épais pour cela : vingt-quatre est donc le mieux possible

# Parcours lab dans le navigateur

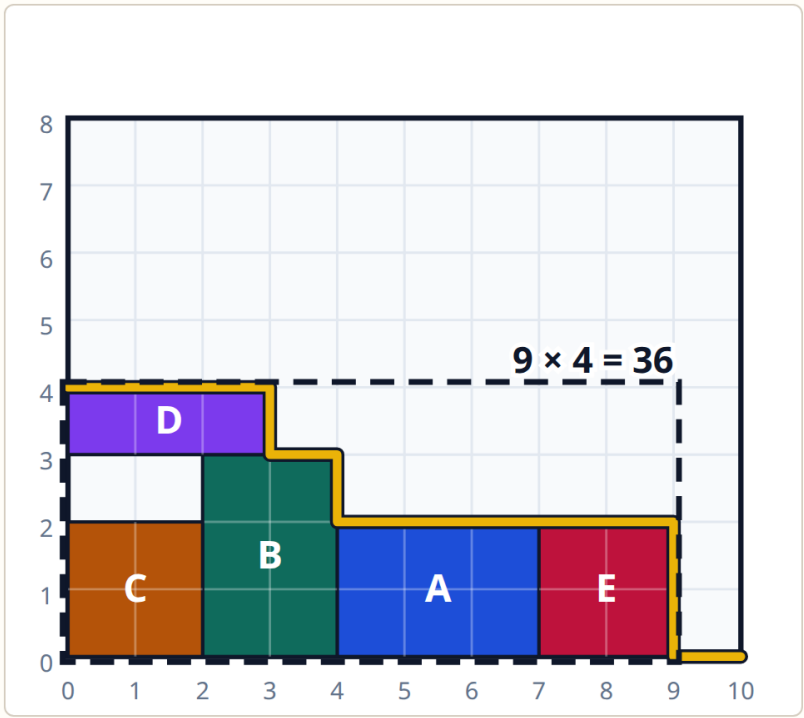
**Arbre et plan de masse**

**L'arbre**




trait plein = à côté (enfant gauche)
trait en pointillé = à l'étage (enfant droit)
petit nombre = ordre de parcours

**Le plan de masse (contour 10 × 8)**




ligne jaune = ligne d'horizon
boîte en pointillé = boîte englobante

Départ du lab dans le navigateur

# Parcours implémentation

wsl — module02-03-bstar-tree/examples/.

```
# Track A — real Unix
# real shell session (Track A)

$ python3 bstar_check.py --show | grep PASS | head -6
PASS  golden A(B(C(E,-),-),D): A (0,0) B (3,0) C (5,0) D (0,2) E (7,0)
PASS  golden: the module list in reverse order gives the same pack
PASS  golden: the pack keeps each block's w and h
PASS  golden: pack_bstar does not change the tree or the module list you pass
PASS  golden: bounding box 9x3 = 27
PASS  golden: legal in the 10x8 outline

$ python3 bstar_check.py --impl wrong/right_first.py | grep FAIL
FAIL  lift C(B(A(E,-),-),D): D lands at (0,3), not (0,2), because B is 3 high
FAIL  order A(B(C(-,E),D),-): left subtree before right child gives a 7x5 box
FAIL  best A(D,C(E(B,-),-)): B is lifted onto D, bounding box 6x4 = 24
FAIL  all 5040 labelled trees on A to E: your pack equals the reference pack
25 PASS, 4 FAIL, 0 SKIP
```

Session de terminal réelle

# Parcours implémentation — à essayer

```
# afficher toutes les bonnes réponses
python3 examples/bstar_check.py --show

# vérifier vos propres fonctions
python3 examples/bstar_check.py --impl my_bstar.py
```

# À retenir

- Enfant gauche : à côté
- Enfant droit : à l'étage, avec le même bord gauche
- Parcourez l'arbre : le nœud, tout son côté gauche, puis son enfant droit
- Chaque bloc tombe jusqu'à toucher la ligne d'horizon
- On note un plan de masse avec sa boîte englobante, et on compte les cases perdues
- Un arbre ne chevauche jamais, mais il peut dépasser : vérifiez le contour

# À vous de jouer

- Choisissez un parcours : le lab dans le navigateur, Python, ou les deux
- Cochez chaque case de la liste de contrôle, puis faites le court quiz
- La suite, c'est la paire de séquences : deux listes de noms au lieu d'un arbre
- Faites une pause et devinez : deux listes décrivent-elles notre plan à vingt-quatre ?
- Oui : deux listes suffisent
- C'est la paire de séquences, dans le prochain lab